

5.1 Pod adresem <http://127.0.0.1:5001> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5001/submit/public> oraz <http://127.0.0.1:5001/submit/private>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5001:5001 --name ex1 docker.io/mazurkatarzyna/gpg-ex1:latest  
podman run -p 5001:5001 --name ex1 docker.io/mazurkatarzyna/gpg-ex1:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5001:5001 --name ex1 ghcr.io/mazurkatarzynaumcs/gpg-ex1:latest  
podman run -p 5001:5001 --name ex1 ghcr.io/mazurkatarzynaumcs/gpg-ex1:latest
```

- (b) Wygeneruj parę kluczy GPG używając algorytmu RSA.
- (c) Wyeksportuj klucz publiczny i klucz prywatny do plików, odpowiednio `pub.key` oraz `priv.key`.
- (d) Wyślij request do endpointa <http://127.0.0.1:5001/submit/public> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz publiczny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.
- (e) Wyślij request do endpointa <http://127.0.0.1:5001/submit/private> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz prywatny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.

UWAGI:

- Aby za pomocą narzędzia `cURL` wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

```
(.python_env) └─[admin@parrot]-[~]  
└─$ gpg --gen-key  
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
  
Note: Use "gpg --full-generate-key" for a full featured key generation dialog.  
  
GnuPG needs to construct a user ID to identify your key.  
  
Real name: Adam Nowak  
Email address: adam@nowak.pl  
You selected this USER-ID:  
  "Adam Nowak <adam@nowak.pl>"  
  
Change (N)ame, (E)mail, or (O)kay/(Q)uit? o  
We need to generate a lot of random bytes. It is a good idea to perform  
some other action (type on the keyboard, move the mouse, utilize the  
disks) during the prime generation; this gives the random number  
generator a better chance to gain enough entropy.  
We need to generate a lot of random bytes. It is a good idea to perform  
some other action (type on the keyboard, move the mouse, utilize the  
disks) during the prime generation; this gives the random number  
generator a better chance to gain enough entropy.  
gpg: revocation certificate stored as '/home/admin/.gnupg/openpgp-revocs.d/602FF61D6B327E2FDCF83875ADFF779EF261F9AE.rev'  
public and secret key created and signed.  
  
pub   rsa3072 2025-11-17 [SC] [expires: 2027-11-17]  
       602FF61D6B327E2FDCF83875ADFF779EF261F9AE  
uid           Adam Nowak <adam@nowak.pl>  
sub   rsa3072 2025-11-17 [E] [expires: 2027-11-17]
```

```
(.python_env) └─[admin@parrot]-[~]
└─ $gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 2 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 2u
gpg: next trustdb check due at 2026-11-12
/home/admin/.gnupg/pubring.kbx
-----
pub  rsa2048 2025-11-12 [SC] [expires: 2026-11-12]
     82E7B3176D404400AA23BFCC6C4D02D44C4E40A6
uid          [ultimate] Jan Paweł 2 (test) <jp2@watican.com>
sub  rsa2048 2025-11-12 [E] [expires: 2026-11-12]

pub  rsa4096 2017-11-30 [SC] [expired: 2021-11-30]
     7FCA9277C31BB06593495B5D228C64831032DFF0
uid          [ expired] Marek Miśkiewicz (Gmail general authorisation.) <marek.miskiewicz@gmail.com>
uid          [ expired] Marek Miśkiewicz <mmiskiewicz@pjwstk.edu.pl>
uid          [ expired] Marek Miśkiewicz <marek.miskiewicz@icloud.com>

pub  rsa2048 2019-04-14 [SC] [expired: 2020-04-13]
     837C745417D1BD209C9096C42AB3C12293B5AD00
uid          [ expired] jan nowak <obfuskacja@wp.pl>

pub  rsa3072 2025-11-17 [SC] [expires: 2027-11-17]
     602FF61D6B327E2FDCF83875ADFF779EF261F9AE
uid          [ultimate] Adam Nowak <adam@nowak.pl>
sub  rsa3072 2025-11-17 [E] [expires: 2027-11-17]
```

```
(.python_env) └─[admin@parrot]-[~]
└─ $gpg --list-secret-keys
/home/admin/.gnupg/pubring.kbx
-----
sec  rsa2048 2025-11-12 [SC] [expires: 2026-11-12]
     82E7B3176D404400AA23BFCC6C4D02D44C4E40A6
uid          [ultimate] Jan Paweł 2 (test) <jp2@watican.com>
ssb  rsa2048 2025-11-12 [E] [expires: 2026-11-12]

sec  rsa3072 2025-11-17 [SC] [expires: 2027-11-17]
     602FF61D6B327E2FDCF83875ADFF779EF261F9AE
uid          [ultimate] Adam Nowak <adam@nowak.pl>
ssb  rsa3072 2025-11-17 [E] [expires: 2027-11-17]
```

```
(.python_env) [admin@parrot]~$ gpg --full-generate-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA (default)
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 1024
Requested keysize is 1024 bits
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months
    <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Adam Nowak
Email address: adamnowak@mail.pl
Comment:
You selected this USER-ID:
    "Adam Nowak <adamnowak@mail.pl>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
```

```
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/admin/.gnupg/openpgp-revocs.d/D5E64E84E5742F0D3B1
0567B72DBF136D1E3400A.rev'
public and secret key created and signed.

pub   rsa1024 2025-11-17 [SC]
       D5E64E84E5742F0D3B10567B72DBF136D1E3400A
uid           Adam Nowak <adamnowak@mail.pl>
sub   rsa1024 2025-11-17 [E]
```

```
(.python_env) [admin@parrot]-[~]  
$gpg --list-keys  
gpg: checking the trustdb  
gpg: marginals needed: 3 completes needed: 1 trust model: pgp  
gpg: depth: 0 valid: 3 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 3u  
gpg: next trustdb check due at 2026-11-12  
/home/admin/.gnupg/pubring.kbx  
-----  
pub   rsa2048 2025-11-12 [SC] [expires: 2026-11-12]  
      82E7B3176D404400AA23BFCC6C4D02D44C4E40A6  
uid           [ultimate] Jan Paweł 2 (test) <jp2@vatican.com>  
sub   rsa2048 2025-11-12 [E] [expires: 2026-11-12]  
  
pub   rsa4096 2017-11-30 [SC] [expired: 2021-11-30]  
      7FCA9277C31BB06593495B5D228C64831032DFF0  
uid           [ expired] Marek Miśkiewicz (Gmail general authorisation.) <marek.miskiewicz@gmail.com>  
uid           [ expired] Marek Miśkiewicz <mmiskiewicz@pjawst.edu.pl>  
uid           [ expired] Marek Miśkiewicz <marek.miskiewicz@icloud.com>  
  
pub   rsa2048 2019-04-14 [SC] [expired: 2020-04-13]  
      837C745417D1BD209C9096C42AB3C12293B5AD00  
uid           [ expired] jan nowak <obfuskacja@wp.pl>  
  
pub   rsa3072 2025-11-17 [SC] [expires: 2027-11-17]  
      602FF61D6B327E2FDCF83875ADFF779EF261F9AE  
uid           [ultimate] Adam Nowak <adam@nowak.pl>  
sub   rsa3072 2025-11-17 [E] [expires: 2027-11-17]  
  
pub   rsa1024 2025-11-17 [SC]  
      D5E64E84E5742F0D3B10567B72DBF136D1E3400A  
uid           [ultimate] Adam Nowak <adamnowak@mail.pl>  
sub   rsa1024 2025-11-17 [E]
```

```
(.python_env) [admin@parrot]-[~]
└─$ gpg --armor --export adam@nowak.pl > pub.key
(.python_env) [admin@parrot]-[~]
└─$ cat pub.key
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQGNBGka6egBDADqiSClS5FqRJ+Lhxb1GKv35z3XrGjrvVfjMGUV907Wqbfd8VYe
ro4EgE+MG9t+BBtDGLRET2PB2zeeQuLYFkopwWU8uSq/EzqIwJmFSoGHbsqRrc7G
wVtlylIq+o55RCBSWcBUUIG2jH+r3SMt7hCx14Q1k1TMNbtUaSlWjRUS3U8U7VfX
M7kqya35jkJEKt76lHxw0oatwx0t8CaoqT1iShBFBP0cVC9JAI87wp1lX0x8705S
3EPVo5YhuN68rN7+5n8cz69twp+xQp1Ym9fGqy1lM8qViuDZGXje+mpwW05IumZd
sSFSDeIdhN/loz408zQ9LZbjzswovACg4VqJkBCn5g4mCn9tBdRxISeWk6zq3wvs
YtXr5p1YKM67Nsp4xCFAWlyQ+C9MwsDceP2xGtL4E/jS/dv4qT9IPwzm0UdxiwYM
60pGj9Q8MpPOKJMRJISUC1Acna/3/JD5lFIGD79z3fSVHPA5VH2Mx+Z6GFCVsVUz
oEC8EzYawRplvA8AEQEAAQQAQRhbSBOb3dhayA8YWRhbUBub3dhay5wbD6JAdQE
EwEKAD4WIQRgL/YdzJ+L9z40Hwt/3ee8mH5rgUCaRrp6AIbAwUJA8JnAAULCQgH
AgYVCgkICwIEFgIDAQIeAQIXgAAKCRct/3ee8mH5roV5C/9IdEu55vLh4UVwTtR5
6QIZwSE64chgdatZ4S9hxyCl3YkG7nNi155PZqtDXjiTPmfk0S2xba0EE9/F2j
utVgWXBuH+bZmcYivTlwg1msnGygS6aDyDs4v6cwh5va8YsYqbKdNjNsSgq1JAqH
P1VuoiFp87B6D8BPdH4Q6EAcmiS5Qpxk0AJ3faY90SY/A0YVYlQvXRbvr9rYvc6f
QSU6WjSn0KrhPz4AfTZ5ev3JVrCTH+uHe0+fkTgHW1uwHr0iHTorunrIoUmVCn4H
d3jttWw8vCnTngg30kZx8k3gFnI0qVoVRkH8Fx8Pb9K4pytoP9uymvg/vtNuB2NU0
X9V/VudsT5JtOwsx/SWKfXooRrhAm/1CLI3rdNdukImvVTPJIKN1spw4xT0q6zH8
0EJtjPZ3Ypnt0FMbS8gsKEsdJfA0ShEDCd16nQ6GsSHb661ew4mNfnkNUbjm2k72
GMUsFfhgirmwJhHwLtoiKfjckK69X2xAn0qELfah5fk+Lpim5AY0EaRrp6AEMAKOL
iSkVjldIKr3JYKBQELQAY9HZg18G/m+Vlj51+pUJ9GRgIJH88NafFzJ/s5VB1p8V
8U+Wgdwo5kQwVBoy9EZr9eHSm0ApXZQzGQNIbLVpgVJSA0XI59rE3D+PvHbgLmMf
Shkyvj9381FxyJCIckDecJlN3RWP6Py2dqAYSISv5QoP5aWRCvS1Tz5G+3ZdYML
oGx22Ph3U0AX0pzgwKYkJOmpsx7uGL551MYv14bck28vVss59lyIMhYWJ3lMrhKm
2oGb5/dLT9E9R5L2ux/9Exg160NdU+mCsec1WkVvAlxxmx9fq0o1B0LubiPtJxsQ
5VI1SNoXJGwtRkUizoPGICEDkQPHGKcULvDCypu+3qb+V18xgHaG/XKyG8ikmuxJ
bNYpMP4KM6jNADqyHweF4KKhTJb03e0pERjqW6081qGhYvFm/k/nEYNJ9TTb3y+E
vc3V7D5E3zRyU7d5+80xxrlZe3WeKpw5G5v+DB8VYSYzQUYg+MDRBG6ExwCt+QAR
AQABiQG8BBgBCgAmFiEEYC/2HwSyfi/c+Dh1rf93nvJh+a4FAmka6egCGwFCQPC
ZwAACGkQrf93nvJh+a5Rzwv/e0EZRvtbq4ShCOifqcPYqVxfh8nPf+zo8TNmvotA
gKWD4htHpN7Y5VXo60x0fwz4vhxwqkoKayUKsJcbwrEbyDqVbUMVMTur0r+0qROD
qxjVxyPCc93qrCGpP0Bw7EZ9L1HHjlKjzqr7Q53BTvHKaphpw19kme0EtMB5bBCO
```



```
(.python_env) └─[admin@parrot]─[~]
└─$gpg --armor --export-secret-key adam@nowak.pl > priv.key
(.python_env) └─[admin@parrot]─[~]
└─$cat priv.key
-----BEGIN PGP PRIVATE KEY BLOCK-----

lQWGBGka6egBDADqiSClS5FqRJ+Lhxb1GKv35z3XrGjrVwFjMGUV907Wqbfd8VYe
ro4EgE+MG9t+BBtDGLRET2PB2zeeQuLYFkopwWU8uSq/EzqIwJmfSoGHbsqRrc7G
wVtlylIq+o55RCBSWcBUUIG2jH+r3SMt7hCx14QlklTMNbtUaSlWjRUS3U8U7VfX
M7kqya35jkJEKt76lHxw0oatwx0t8CaoqT1iShBFBP0cVC9JAI87wp1lX0x8705S
3EPVo5YhuN68rN7+5n8cz69twp+xQpLYm9fGqyl1M8qViuDZGXje+mpwW05IumZd
sSFSDeIdhN/loz408zQ9LZbjzswovACg4VqJkBCn5g4mCn9tBdRxISeWk6zq3wvs
YtXr5p1YKM67Nsp4xCFAWlyQ+C9MwsDceP2xGtL4E/jS/dv4qT9IPwzm0UdxiwYM
60pGj9Q8MpPOKJMRJISUC1Acna/3/JD5lFIgD79z3fSVHPA5VH2Mx+Z6GFCVsVUz
oEC8EzYaWRPlvA8AEQEAAf4HAWLu4Wu8W0o0Lf9jbM2LlMBF5ytjJsoeaJaQvXrJ
drCsFEOwTEZJ+C56qvQfmdX4uc5+cKMpAJRbmco10zd+0qMr2BC/wB6dojTULOW
oZ2aAZFQkXw63olJlY8cE9A/MkbwqETyxkkEzG+yHvWAAXiKN9nN2Mt/UeCmOXIZ
Tt9zwJrYdF5qxQ3f+5V1gANeK80a6J+tyBfTaK6Z1UPAg0dqWyu1NF/+480adlsb
PQOA2hINmqXAQ8VYw24wzIXjIneZby6iFYyLurzuk55L9YGrW4RmabMiE62P08Cr
Uw1QkeySG2Ui72IyYQJB0us5tvz1bD4epkGf0rQRUEQKpc0LWU0d+/1tNvraqbs
LUGJvR7y1jJ2VZ/pBNWL9a4EZbuv//LUl/F4YSHiHJcw8zJHP1X2JtywTBr03V0B
d+eU5grq4MS6+KL1lNFkiaVUiLAhTTY9Axpcktpeb2fgLKeYCYub6G72iWDD0lK/
Ma/nSBAtW4o3yjcRUsmP/2KUjoM2n9N2EujGjn+4dk22++odaMxP4NIR4mE6rmGx
t9mTNRANVPQhg5oDg+5p+BIgf56JZJqFx10cGAeBKHA7StpfIIccJzAdReQxS17u
J7Vy4NR3DKHIbesczA17T6TX+YgZ0fJcqDy9uW9F05UVoLIyAHP5YDepcQy5o0by
oku8TEYdVFb0/QMGFOEomqARfFqJ6XipGCNyyBiMl0Lsoi86LZz2669CJl2YaroR
lGPIY+73QMt49twd3IuFUqytag95mzXREJTUnIHwx2ASdv8Vv0kJKtrxc3IT2gOp
RunBXW0ezt01MHboV0uldcgJo9Mzu9f1JnQdS1CHQ157QoBhHlPrYaI9h9e7vbxL
b3CyTn+Q5XatLIIdqcH0g1nSLDv7Z8KQBIFlKQvZmSm1TIWI7G9nWNaxP6Z8QlDl0
cmYnHnjWUKsNnBTdC7X2DHBfcU3tFykPBINJswQVZNQqgrG3fR3oYDNdVK58zloh
A6KFgZmS2tjjVp4TD1MUIl5vKt+5s0giMhb/lwCkE/xKHwMCLTrJotMaaP7eEUlE
WCPFd1A3RvZMdBpAqxTDYjq6weZsr1/0SnTCMyPabJmgr5kfX319dc0FFT0TGkJ
ILGlkjn/0gb5t9DBFYQ2qtS1m9MT3rc1Vaem0Nhv8B72hyU1u5jIw+5yNK352KIo
ezXRfWqOw3N+mA5VyR5WnepFTSgCzNYujVz+Vi0YbvNSuMvI9YqueeyryPoFUBWe
JUk8sMBmobYEHhf2Q4B/+gUniENe3oyH8RPYpn7ZmQfZhGov06oHYM/DND7uKMYy
BPCT+Mau7/nC32KMHGiBm4e0HAY02reTKlQaQWRhbSR0b3dhavA8YWRhbUBub3dh
```



```
=mhvB
-----END PGP PRIVATE KEY BLOCK-----
(.python_env) [admin@parrot]~
└─ $curl -X POST http://localhost:5001/submit/private -F "file=@priv.key"
{
  "algo": "1",
  "created": "1763371496",
  "expires": "1826443496",
  "fingerprint": "602FF61D6B327E2FDCF83875ADFF779EF261F9AE",
  "key_id": "ADFF779EF261F9AE",
  "length": "3072",
  "trust": "-",
  "type": "private",
  "uids": [
    "Adam Nowak <adam@nowak.pl>"
  ]
}
(.python_env) [admin@parrot]~
└─ $curl -X POST http://localhost:5001/submit/public -F "file=@pub.key"
{
  "algo": "1",
  "created": "1763371496",
  "expires": "1826443496",
  "fingerprint": "602FF61D6B327E2FDCF83875ADFF779EF261F9AE",
  "key_id": "ADFF779EF261F9AE",
  "length": "3072",
  "trust": "-",
  "type": "public",
  "uids": [
    "Adam Nowak <adam@nowak.pl>"
  ]
}
(.python_env) [admin@parrot]~
```

5.2 Pod adresem <http://127.0.0.1:5002> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5002/submit/public> oraz <http://127.0.0.1:5002/submit/private>.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5002:5002 --name ex2 docker.io/mazurkatarzyna/gpg-ex2:latest
podman run -p 5002:5002 --name ex2 docker.io/mazurkatarzyna/gpg-ex2:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5002:5002 --name ex2 ghcr.io/mazurkatarzynaumcs/gpg-ex2:latest
podman run -p 5002:5002 --name ex2 ghcr.io/mazurkatarzynaumcs/gpg-ex2:latest
```

- (b) Wygeneruj parę kluczy GPG używając algorytmu DSA.
- (c) Wyeksportuj klucz publiczny i klucz prywatny do plików, odpowiednio `pub.key` oraz `priv.key`.
- (d) Wyślij request do endpointa <http://127.0.0.1:5002/submit/public> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz publiczny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.
- (e) Wyślij request do endpointa <http://127.0.0.1:5002/submit/private> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz prywatny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.

UWAGI:

- Aby za pomocą narzędzia `cURL` wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

```
(.python_env) [admin@parrot]~  
$ gpg --full-generate-key  
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
  
Please select what kind of key you want:  
  (1) RSA and RSA (default)  
  (2) DSA and Elgamal  
  (3) DSA (sign only)  
  (4) RSA (sign only)  
  (14) Existing key from card  
Your selection? 3  
DSA keys may be between 1024 and 3072 bits long.  
What keysize do you want? (2048) 1024  
Requested keysize is 1024 bits  
Please specify how long the key should be valid.  
  0 = key does not expire  
  <n> = key expires in n days  
  <n>w = key expires in n weeks  
  <n>m = key expires in n months  
  <n>y = key expires in n years  
Key is valid for? (0) 1  
Key expires at Tue 18 Nov 2025 04:42:56 AM EST  
Is this correct? (y/N) y  
  
GnuPG needs to construct a user ID to identify your key.  
  
Real name: Adam Kowalski  
Email address: adam@kowal.pl  
Comment:  
You selected this USER-ID:  
  "Adam Kowalski <adam@kowal.pl>"  
  
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit?  
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o  
We need to generate a lot of random bytes. It is a good idea to perform  
some other action (type on the keyboard, move the mouse, utilize the
```

some other action (type on the keyboard, move the mouse, utilize the disks) during the prime generation; this gives the random number generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/admin/.gnupg/openpgp-revocs.d/A79DCD1C6E61073982D5E7BA36FEC8C33AA9F47.rev'
public and secret key created and signed.

Note that this key cannot be used for encryption. You may want to use the command "--edit-key" to generate a subkey for this purpose.

```
pub   dsa1024 2025-11-17 [SC] [expires: 2025-11-18]
       A79DCD1C6E61073982D5E7BA36FEC8C33AA9F47
uid           Adam Kowalski <adam@kowal.pl>
```

```
(.python_env) [admin@parrot]~[~]
└─ $gpg --armor --export-key A79DCD1C6E61073982D5E7BA36FEC8C33AA9F47 > pub.key
invalid option "--export-key"
(.python_env) [x]~[admin@parrot]~[~]
└─ $gpg --armor --export A79DCD1C6E61073982D5E7BA36FEC8C33AA9F47 > pub.key
(.python_env) [admin@parrot]~[~]
└─ $cat pub.key
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQGibGka7jURBACTLr0XcJqMIChiutyhvnRzkGus0T3HwXBg1qMtiYBKLTASx4a0
wSs+7G34o8eD9cWmWB1sHx4MC5D7gz1nkOUCXEai7gn/T1VHd0l+WF00hSHCv9pq
os0+1mTgz4t8NDWh/1IswgvTEXpz6zK/UEf/Bo3ZhYyLM9WVai5JzZQ09wCgj0pt
BDGtnd1q7iahRUu9CPKEgUkEAI2MPYFXbPYuav74mtM3N+PJi3BgmFB3ZnkHqQ5o
7IXE6NrwoXwBK4AByztIiF66CfMhqfvlis7MV0AEYpM0WefdC/P7p6C+j7skhuth
jwYl3upK/Bwwyfz7x8UF65hU10phuiA+o6gVxIpVcaqQsZuvSMr+/XzR5o81Vd7
6BRsA/w03fDgCHamsu4ziHgWh6lJWA2VzqmCe0bSachlKjauQ6yYZj3DWdek27QN
2XSJDRBv1M2R3o3TWIEoHzuRI447Hry+uDnqmmStu/JC0xJYhjIGqzz0pEbUTjHJ
73IVQKMoxg/CxRuPLuSB43YbIWbm9HdDec3q5RDhXkwoQQY20LQdQWRhbSBLb3dh
bHNraSA8YWRhbUBrb3dhbC5wbD6IfgQTEQIAPhYhBKedzRxuYQc5gtXnujb+zYwz
qp9HBQJpGu41AhsDBQkAAVgABQsJCAcCBhUKCQgLAgQWAgMBAh4BAheAAAJEDb+
zYwzqp9HsbsAn0A4G6N63ae8RAtxd44wWX4VftbSAJ0SuGWVb8IgekkRghCZidH+
g8Hu3Q==
=R3aj
-----END PGP PUBLIC KEY BLOCK-----
(.python_env) [admin@parrot]~[~]
```

```

(.python_env) └─[admin@parrot]-[~]
└─ $gpg --armor --export-secret-key A79DCD1C6E61073982D5E7BA36FECD8C33AA9F47 > priv.key
(.python_env) └─[admin@parrot]-[~]
└─ $cat priv.key
-----BEGIN PGP PRIVATE KEY BLOCK-----

lQG7BGka7jURBACTLr0XcJqMIChiutyhvnRzkGus0T3HwXBg1qMtiYBKLTASx4a0
wSs+7G34o8eD9cWmWB1sHx4MC5D7gz1nk0UCXEai7gn/T1Vhd0l+WF00hSHCv9pq
os0+1mTgz4t8NDWh/lIswgvTEXpz6zK/UEf/Bo3ZhYyLM9wVai5JzZQ09wCgj0pt
BDGtnd1q7iahRUu9CPKEgUkEAI2MPYFXbPYuav74mtM3N+PJi3BgmFB3ZnkHqq5o
7IXE6NrwoXBK4AByztIiF66CfMhqfvliS7MV0AEYpM0WefdC/P7p6C+j7skhuth
jwYlZ3upK/BwwyFz7x8UF65hU10phuiA+o6gVxIpVcaqQsZuvSMr+/XzR5o81Vd7
6BRsA/w03fDgCHAMsu4ziHgWh6lJWA2VzqmCeObSAch1KjauQ6yYZj3DWdek27QN
2XSJDRBv1M2R3c3TWIEoHzuRI447Hry+uDnqmmStu/JC0xJYhjIGqzz0pEbUTjHJ
73IVQKMoXg/CxRuPLuSB43YbIWbm9HdDec3q5RDhXkwoQQY20AAAniW7eYboSb9m
sBU7Tdo8nsQIIIVf1Cg20HUFkYW0gS293YWxza2kgPGFkYW1Aa293YWwucGw+iH4E
ExECAD4WlQSNnc0cbmEH0YL57o2/s2MM6qfRwUCaRruNQIbAwUJAAFRgAULCQgH
AgYVCgkICwIEFgIDAQIEAQIXgAAKCRA2/s2MM6qfR7G7AJ9A0Bujet2nvEQLcXe0
MFl+FX7W0gCdErhl1W/CIHpJEYIQmYnR/oPB7t0=
=N04S
-----END PGP PRIVATE KEY BLOCK-----

(.python_env) └─[admin@parrot]-[~]
└─ $curl -X POST http://localhost:5002/submit/public -F "file=@pub.key"
{
  "algorithm": "DSA",
  "created": "1763372597",
  "expires": "1763458997",
  "fingerprint": "A79DCD1C6E61073982D5E7BA36FECD8C33AA9F47",
  "format": "GPG/OpenPGP",
  "key_id": "36FECD8C33AA9F47",
  "key_size": 1024,
  "trust": "-",
  "type": "public",
  "uids": [
    "Adam Kowalski <adam@kowal.pl>"
  ]
}
(.python_env) └─[admin@parrot]-[~]

```

```
(.python_env) [admin@parrot]~$ curl -X POST http://localhost:5002/submit/private -F "file=@priv.key"
{
  "algorithm": "DSA",
  "created": "1763372597",
  "expires": "1763458997",
  "fingerprint": "A79DCD1C6E61073982D5E7BA36FECD8C33AA9F47",
  "format": "GPG/OpenPGP",
  "key_id": "36FECD8C33AA9F47",
  "key_size": 1024,
  "trust": "-",
  "type": "private",
  "uids": [
    "Adam Kowalski <adam@kowal.pl>"
  ]
}
```

5.3 Pod adresem <http://127.0.0.1:5003> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5003/submit/public> oraz <http://127.0.0.1:5003/submit/private>.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5003:5003 --name ex3 docker.io/mazurkatarzyna/gpg-ex3:latest
podman run -p 5003:5003 --name ex3 docker.io/mazurkatarzyna/gpg-ex3:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5003:5003 --name ex3 ghcr.io/mazurkatarzynaumcs/gpg-ex3:latest
podman run -p 5003:5003 --name ex3 ghcr.io/mazurkatarzynaumcs/gpg-ex3:latest
```

- (b) Wygeneruj parę kluczy GPG używając algorytmu RSA. Klucze powinny mieć 1024 bity długości, mają być ważne rok, oraz być wygenerowane dla adresu `student@uczelnia.pl`.
- (c) Wyeksportuj klucz publiczny i klucz prywatny do plików, odpowiednio `pub.key` oraz `priv.key`.
- (d) Wyślij request do endpointa <http://127.0.0.1:5003/submit/public> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz publiczny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.
- (e) Wyślij request do endpointa <http://127.0.0.1:5003/submit/private> używając metody HTTP POST i prześlij na serwer wyeksportowany klucz prywatny GPG. Serwer w odpowiedzi zwróci informacje o kluczu.

```
(.python_env) [admin@parrot]~  
└─$ gpg --full-generate-key  
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
  
Please select what kind of key you want:  
  (1) RSA and RSA (default)  
  (2) DSA and Elgamal  
  (3) DSA (sign only)  
  (4) RSA (sign only)  
  (14) Existing key from card  
Your selection? 1  
RSA keys may be between 1024 and 4096 bits long.  
What keysize do you want? (3072) 1024  
Requested keysize is 1024 bits  
Please specify how long the key should be valid.  
    0 = key does not expire  
    <n> = key expires in n days  
    <n>w = key expires in n weeks  
    <n>m = key expires in n months  
    <n>y = key expires in n years  
Key is valid for? (0) 1y  
Key expires at Tue 17 Nov 2026 04:49:16 AM EST  
Is this correct? (y/N) y  
  
GnuPG needs to construct a user ID to identify your key.  
  
Real name: student  
Email address: student@uczelnia.pl  
Comment:  
You selected this USER-ID:  
    "student <student@uczelnia.pl>"  
  
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o  
We need to generate a lot of random bytes. It is a good idea to perform  
some other action (type on the keyboard, move the mouse, utilize the  
disks) during the prime generation; this gives the random number
```

```
generator a better chance to gain enough entropy.  
gpg: revocation certificate stored as '/home/admin/.gnupg/openpgp-revocs.d/DF0F9B6579736BE0EB1F40AE9D7AAA6702B313E8.rev'  
public and secret key created and signed.
```

```
pub  rsa1024 2025-11-17 [SC] [expires: 2026-11-17]  
      DF0F9B6579736BE0EB1F40AE9D7AAA6702B313E8  
uid          student <student@uczelnia.pl>  
sub  rsa1024 2025-11-17 [E] [expires: 2026-11-17]
```

```
(.python_env) └─[admin@parrot]-[~]  
└─ $gpg --armor --export DF0F9B6579736BE0EB1F40AE9D7AAA6702B313E8 > pub.key  
(.python_env) └─[admin@parrot]-[~]  
└─ $gpg --armor --export-secret-key DF0F9B6579736BE0EB1F40AE9D7AAA6702B313E8 > priv.key  
(.python_env) └─[admin@parrot]-[~]  
└─ $head pub.key | head  
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

```
mI0EaRrvvQEEAKmkjYRxlb1r9twlS6wPbrjGwHbfYjUpUXwYVYy4sN8GFkPLA7x2  
Kyl4erPiAzAr5AJERaPH/iNZrteqaec585NNPF4KKb6iXvek95Bv9GDhPzw/RqnY  
Kz269QV/wnyuzL0E/AhtEcr3bvQtG2eFWVaPy94kuyGZzR9e9PoyyfmfABEBAAg0  
HXN0dWRlbnQgPHN0dWRlbnRAdWN6ZWxuaWEucGw+iNQEEwEKAD4WIQTfD5tleXNr  
40sfQK6deqpnArMT6AUCaRrvvQIbAwUJAeEzgAULCQgHAgYVCgkICwIEFgIDAQIe  
AQIXgAAKCRCdeqpnArMT6PLsBACb33NZLe6rcNS8MOZP7sCPcTVLe0h6t9Qau037  
f7uvsrdi0cvPMmdS499OVUdrZL7ZDEf04Mno7Priczc2pfdogJ2l4YSNlu+le+VR  
1Z3CAjz2XKjfgSaz6y4xYuCqBsi57FE3zBD91UzHj0ssUYD76bCLsxmzbSt0HcFl
```

```
(.python_env) └─[admin@parrot]-[~]  
└─ $head priv.key | head  
-----BEGIN PGP PRIVATE KEY BLOCK-----
```

```
lQHYBGka770BBACppI2EcZW9a/bcJUusD264xsB232I1KVF8GGFcULdfBhZDyw08  
dispeHqz4gMwK+QCREWjx/4jWa7XqmnnOf0TTTxeCim+ol73pPeQb/Rg4T88P0ap  
2Cs9uvUFf8J8rsy9BPwIbRHK9270LRtnhVlWj8veJLshmc0fXvT6MsnzHwARAQAB  
AAP7Ba3VKlFjyEjbl0Fpe356w5VXvRhYGgbw6HJSfr9qEiqY8aiaSiMpBfbiRuly  
iVHomt8LARwdypfxKjJVNZnsNvyxmIotVowIiwoTyIB8B2dw10x3MEpkaabI0L9z  
QzOg9pPvPMm4zp2PHoYmaBABtWT66Vxvrf/13UDhv2CKrECAMOL3lX1PjVnlNdj  
QBHZ90XAoKFCty77fAizuHXlBvjy4ywcVmPEv3xnmKX9vIWgPql8zPdMt94HSUnu  
BPkaF48CAN4WmMQLCG/SM0xb0Pfo3qRIcCy6md4FY3GFpcyzNEW1tNgVxbjPTArX
```



```
(.python_env) └─[x]─[admin@parrot]─[~]  
└─ $curl -X POST http://localhost:5003/submit/public -F "file=@pub.key"  
{  
  "algo": "1",  
  "created": "1763372989",  
  "expires": "1794908989",  
  "fingerprint": "DF0F9B6579736BE0EB1F40AE9D7AAA6702B313E8",  
  "key_id": "9D7AAA6702B313E8",  
  "length": "1024",  
  "trust": "-",  
  "type": "public",  
  "uids": [  
    "student <student@uczelnia.pl>"  
  ],  
  "validation": "passed"  
}  
(.python_env) └─[admin@parrot]─[~]  
└─ $curl -X POST http://localhost:5003/submit/private -F "file=@priv.key"  
{  
  "algo": "1",  
  "created": "1763372989",  
  "expires": "1794908989",  
  "fingerprint": "DF0F9B6579736BE0EB1F40AE9D7AAA6702B313E8",  
  "key_id": "9D7AAA6702B313E8",  
  "length": "1024",  
  "trust": "-",  
  "type": "private",  
  "uids": [  
    "student <student@uczelnia.pl>"  
  ],  
  "validation": "passed"  
}  
(.python_env) └─[admin@parrot]─[~]
```

5.4 Pod adresem <http://127.0.0.1:5004> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5004/decrypt> oraz <http://127.0.0.1:5004/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5004:5004 --name ex4 docker.io/mazurkatarzyna/gpg-ex4:latest
podman run -p 5004:5004 --name ex4 docker.io/mazurkatarzyna/gpg-ex4:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5004:5004 --name ex4 ghcr.io/mazurkatarzynaumcs/gpg-ex4:latest
podman run -p 5004:5004 --name ex4 ghcr.io/mazurkatarzynaumcs/gpg-ex4:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5004/decrypt> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum.
- (d) Odszyfruj plik `encrypted.txt` używając algorytmu CAMELLIA128 i hasła znajdującego się w pliku `passphrase.txt`.
- (e) Wyślij request do endpointa <http://127.0.0.1:5004/submit/> używając metody HTTP POST, i prześlij do serwera ID sesji (jako `session_id`, które znajdziesz w pliku `session_id.txt`) oraz odszyfrowane słowo (jako `decrypted_text`). Serwer w odpowiedzi zwróci informacje o poprawnym lub niepoprawnym deszyfrowaniu.

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji `-o`.
- Podczas deszyfrowania symetrycznego sprawdź opcje:
`--cipher-algo`
`--batch --yes`
`--passphrase`

```
(.python_env) └─[admin@parrot]-[~]  
└─ $curl -s -o zad54.zip -X GET http://localhost:5004/decrypt  
(.python_env) └─[admin@parrot]-[~]  
└─ $ls | grep zad54  
zad54.zip  
(.python_env) └─[admin@parrot]-[~]  
└─ $unzip zad54.zip  
Archive:  zad54.zip  
  inflating: encrypted.txt  
  inflating: passphrase.txt  
  inflating: session_id.txt  
(.python_env) └─[admin@parrot]-[~]  
└─ $cat encrypted.txt  
-----BEGIN PGP MESSAGE-----  
  
jA0ECwMKWqhZNpJv4a//0jsBKZibBzodDVNgftfcdmSG8GTJ/r/dp9rf63lK3eYb  
IBn29w+2IF4Mr7tmAAZ8cvdpeGx5gk7bqix/ig==  
=YYSU  
-----END PGP MESSAGE-----  
(.python_env) └─[admin@parrot]-[~]  
└─ $cat passphrase.txt  
master(.python_env) └─[admin@parrot]-[~]  
└─ $cat session_id.txt  
53b97075c7acb75cbeaec25b9c3f9f07(.python_env) └─[admin@parrot]-[~]  
.
```

```
(.python_env) └─[admin@parrot]-[~]
└─$gpg --cipher-algo CAMELLIA128 --decrypt encrypted.txt
gpg: CAMELLIA128.CFB encrypted data
gpg: cancelled by user
gpg: encrypted with 1 passphrase
gpg: decryption failed: No secret key
(.python_env) └─[x]-[admin@parrot]-[~]
└─$gpg --cipher-algo CAMELLIA128 --decrypt encrypted.txt
gpg: CAMELLIA128.CFB encrypted data
gpg: encrypted with 1 passphrase
123456(.python_env) └─[admin@parrot]-[~]
└─$gpg --cipher-algo CAMELLIA128 --decrypt encrypted.txt > decrypted.txt
gpg: CAMELLIA128.CFB encrypted data
gpg: encrypted with 1 passphrase
(.python_env) └─[admin@parrot]-[~]
└─$cat decrypted.txt
123456(.python_env) └─[admin@parrot]-[~]
└─$curl -X POST http://localhost:5004/submit -H "Content-Type: application/json" -d '{"session_id": "53b97075c7acb75cbeaec25b9c3f9f07", "decrypted_text": "123456"}'
{
  "message": "Gratulacje! Poprawnie odszyfrowa\u0142e\u015b plik!",
  "passphrase_used": "master",
  "secret_word": "123456",
  "status": "success"
}
```

5.5 Pod adresem <http://127.0.0.1:5005> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5005/encrypt> oraz <http://127.0.0.1:5005/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5005:5005 --name ex5 docker.io/mazurkatarzyna/gpg-ex5:latest
podman run -p 5005:5005 --name ex5 docker.io/mazurkatarzyna/gpg-ex5:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5005:5005 --name ex5 ghcr.io/mazurkatarzynaumcs/gpg-ex5:latest
podman run -p 5005:5005 --name ex5 ghcr.io/mazurkatarzynaumcs/gpg-ex5:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5005/encrypt> używając metody HTTP GET. W odpowiedzi od serwera otrzymasz słowo do zaszyfrowania (jako `word_to_encrypt`), hasło potrzebne do szyfrowania (jako `passphrase`) oraz id sesji (jako `session_id`).
- (c) Zaszyfruj odebrane od serwera słowo (`word_to_encrypt`) za pomocą algorytmu AES128 oraz odebranego od serwera hasła (`passphrase`). Zaszyfrowane słowo ma być również zakodowane przy pomocy kodowania base64.
- (d) Wyślij request do endpointa <http://127.0.0.1:5005/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz zaszyfrowany i zakodowany plik (jako plik, `encrypted_file`).

UWAGI:

- Aby wykorzystać kodowanie base64 podczas szyfrowania, użyj opcji `--armor`,
- Aby za pomocą narzędzia curl wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

```
(.python_env) └─[admin@parrot]─[~]
└─ $curl -s -o data.txt -X GET http://127.0.0.1:5005/encrypt
(.python_env) └─[admin@parrot]─[~]
└─ $cat data.txt
{
  "session_id": "dd8413328ab9e2f31e117c1b28d8f137",
  "word_to_encrypt": "leslie",
  "passphrase": "carole",
  "algorithm": "AES128",
  "instructions": "Użyj GPG do zaszyfrowania słowa algorytmem AES128 z podanym hasłem. Następnie wyślij zaszyfrowany plik do /submit wraz z session_id."
}
(.python_env) └─[admin@parrot]─[~]
└─ $echo -n "leslie" > decrypted.txt
(.python_env) └─[admin@parrot]─[~]
└─ $cat decrypted.txt
leslie
(.python_env) └─[admin@parrot]─[~]
```

```
(.python_env) └─[x]─[admin@parrot]─[~]
└─ $gpg --symmetric --cipher-algo AES128 --armor --output encrypted.txt decrypted.txt
File 'encrypted.txt' exists. Overwrite? (y/N) y
(.python_env) └─[admin@parrot]─[~]
└─ $cat encrypted.txt
-----BEGIN PGP MESSAGE-----

jA0EBwMCnarkWlMz4JH/0kgBtXedBSOpWxd0MvPZRxl7VdZt+iYRfx0HEkEjMSGy
98qfkUdA3HUF0sCuiIWH5//H1pWMAtwdNAkQxC6sR/Bs0G/cORghLAs=
=8RCS
-----END PGP MESSAGE-----
```

```
(.python_env) └─[admin@parrot]─[~]
└─ $curl -X POST http://localhost:5005/submit -F "encrypted_file=@encrypted.txt" -F "session_id=dd8413328ab9e2f31e117c1b28d8f137"
{
  "status": "success",
  "message": "Gratulacje! Poprawnie zaszyfrowałeś plik!",
  "word": "leslie",
  "passphrase_used": "carole"
}
(.python_env) └─[admin@parrot]─[~]
```

5.6 Pod adresem `http://127.0.0.1:5006` działa prosty serwer HTTP udostępniający 2 endpointy: `http://127.0.0.1:5006/encrypt` oraz `http://127.0.0.1:5006/submit`.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5006:5006 --name ex6 docker.io/mazurkatarzyna/gpg-ex6:latest
podman run -p 5006:5006 --name ex6 docker.io/mazurkatarzyna/gpg-ex6:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5006:5006 --name ex6 ghcr.io/mazurkatarzynaumcs/gpg-ex6:latest
podman run -p 5006:5006 --name ex6 ghcr.io/mazurkatarzynaumcs/gpg-ex6:latest
```

- (b) Wyślij request do endpointa `http://127.0.0.1:5006/encrypt` używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem `*.zip`.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do zaszyfrowania (`word.txt`).
- (d) Zaimportuj do swojego zbioru kluczy klucz publiczny pobrany od serwera.
- (e) Wykorzystując narzędzie `gpg` oraz pobrany klucz publiczny (z pliku `public_key.asc`), zaszyfruj odebrane od serwera słowo (podczas szyfrowania nie używaj hasła).
- (f) Zaszyfrowane słowo zakoduj w base64 i zapisz do pliku.
- (g) Wyślij request do endpointa `http://127.0.0.1:5006/submit` używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz zaszyfrowane i zakodowane słowo (jako plik, `encrypted_file`).

```
(.python_env) └─[admin@parrot]─[~]
└─$ curl -s -o pliki.zip -X GET http://localhost:5006/encrypt
(.python_env) └─[admin@parrot]─[~]
└─$ unzip pliki.zip
Archive: pliki.zip
  inflating: word.txt
  inflating: public_key.asc
  inflating: private_key.asc
replace session_id.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: session_id.txt
(.python_env) └─[admin@parrot]─[~]
└─$ gpg --import public_key.asc
gpg: key B506DA6CA7ADE60F: public key "Challenge User <challengex6@example.com>" imported
gpg: Total number processed: 1
gpg:      imported: 1
(.python_env) └─[admin@parrot]─[~]
└─$ gpg --list-keys | grep challenge
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid: 5  signed: 0  trust: 0-, 0q, 0n, 0m, 0f, 5u
gpg: next trustdb check due at 2025-11-18
(.python_env) └─[x]─[admin@parrot]─[~]
└─$ gpg --list-keys | grep "challenge"
(.python_env) └─[x]─[admin@parrot]─[~]
└─$ gpg --list-keys
```

```
pub    rsa2048 2025-11-17 [SCEA]
       86E873C0BCAAEF86EA0EB60FB506DA6CA7ADE60F
uid          [ unknown] Challenge User <challengex6@example.com>
```

```
(.python_env) └─[admin@parrot]─[~]
└─ $gpg --encrypt --recipient 86E873C0BCAAEF86EA0EB60FB506DA6CA7ADE60F --armor --output encr
ypted.txt word.txt
gpg: B506DA6CA7ADE60F: There is no assurance this key belongs to the named user

pub  rsa2048/B506DA6CA7ADE60F 2025-11-17 Challenge User <challengex6@example.com>
     Primary key fingerprint: 86E8 73C0 BCAA EF86 EA0E B60F B506 DA6C A7AD E60F

It is NOT certain that the key belongs to the person named
in the user ID.  If you *really* know what you are doing,
you may answer the next question with yes.

Use this key anyway? (y/N) y
File 'encrypted.txt' exists. Overwrite? (y/N) y
(.python_env) └─[admin@parrot]─[~]
└─ $cat encrypted.txt
-----BEGIN PGP MESSAGE-----

hQEMA7UG2mynreYPAQf/V/aE1Q3VNbpPCwvqmx5CjcbDsRhVKXmWtnnNlMXy1Z/E
B0VNf0JvOMDKedKvSLmQT5N5V1ccUlwP1WvA6Pbj6hfIAEek8nS5MndLinlg99qd
xA6m+Khy0bQAuWbT/VF1Tpbv7pm0eU1itNo/UrP7pqkxMWRaCH3TxJjAsxbkeRtY
slhG03qgM8gFn+yejg5D9K/2taUKAxU7tw0QYgxfHl0Z+1+XiwAa0qj1U5j8uD4+
RE1JUsc6PYYqN+S14MVnIMq21Z60iTXCD7eopueSBxcMSeVpjs7JY7PCAjSh0TZ9
+I77yqW+kgmLhB+o/r4V1LdagVklMFJ0Tl0aSQnndJJARW+JeFeoCSTiUn71ME3
mrtI52mveZ+8IvLX+gV2PWDQvpy+D0ortlyOGzPR4at8jVfrLk6s6AyPXZ/JTEQ4
E/YljgatA6tHDw==
=Rz9Q
-----END PGP MESSAGE-----
(.python_env) └─[admin@parrot]─[~]
└─ $curl -X POST http://localhost:5006/submit -F "session_id=$(cat session_id.txt)" -F "encr
ypted_file=@encrypted.txt"
{
  "status": "success",
  "message": "Gratulacje! Poprawnie zaszyfrowałeś plik!",
  "word": "mozart"
}(.python_env) └─[admin@parrot]─[~]
```

5.7 Pod adresem <http://127.0.0.1:5007> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5007/decrypt> oraz <http://127.0.0.1:5007/submit>.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5007:5007 --name ex7 docker.io/mazurkatarzyna/gpg-ex7:latest
podman run -p 5007:5007 --name ex7 docker.io/mazurkatarzyna/gpg-ex7:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5007:5007 --name ex7 ghcr.io/mazurkatarzynaumcs/gpg-ex7:latest
podman run -p 5007:5007 --name ex7 ghcr.io/mazurkatarzynaumcs/gpg-ex7:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5007/decrypt> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do odszyfrowania (`encrypted_word.asc`).
- (d) Odszyfruj odebrane od serwera słowo (`encrypted_word.asc`) za pomocą pobranego klucza prywatnego (`private_key.asc`).
- (e) Wyślij request do endpointa <http://127.0.0.1:5007/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz odszyfrowane słowo (jako tekst, `decrypted_word`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji `-o`


```
(.python_env) └─[admin@parrot]-[~]
└─ $curl -s -o pliki.zip -X GET http://localhost:5007/decrypt
(.python_env) └─[admin@parrot]-[~]
└─ $unzip pliki.zip
Archive: pliki.zip
replace public_key.asc? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: public_key.asc
  inflating: private_key.asc
  inflating: encrypted_word.asc
  inflating: session_id.txt
  inflating: README.txt
(.python_env) └─[admin@parrot]-[~]
└─ $cat README.txt
GPG Decryption Challenge
=====

1. Użyj klucza prywatnego (private_key.asc) do odszyfrowania pliku encrypted_word.asc
2. Wyślij odszyfrowane słowo na endpoint /submit wraz z session_id

Session ID: ee81fc60f523071d1fddb79d9f7848c9
(.python_env) └─[admin@parrot]-[~]
└─ $gpg --import private_key.asc
gpg: key 7C775A70CE558C15: public key "Decrypt Challenge <decrypt@example.com>" imported
gpg: key 7C775A70CE558C15: secret key imported
gpg: Total number processed: 1
gpg:      imported: 1
gpg:      secret keys read: 1
gpg:      secret keys imported: 1
(.python_env) └─[admin@parrot]-[~]
└─ $gpg --decrypt encrypted_word.asc
gpg: encrypted with 2048-bit RSA key, ID 7C775A70CE558C15, created 2025-11-17
    "Decrypt Challenge <decrypt@example.com>"
erenity(.python_env) └─[admin@parrot]-[~]
└─ $echo -n "erenity" > decrypted_word.txt
```

```
└─ $cat session_id.txt
ee81fc60f523071d1fddb79d9f7848c9(.python_env) └─[admin@parrot]-[~]
```

```
└─ $curl -X POST http://localhost:5007/submit -F "session_id=ee81fc60f523071d1fddb79d9f7848c
9" -F "decrypted_word=erenity"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie odszyfrowałeś słowo!",
  "word": "erenity"
}(.python_env) └─[admin@parrot]-[~]
```

5.8 Pod adresem <http://127.0.0.1:5008> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:5008/sign> oraz <http://127.0.0.1:5008/submit>.

- (a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 5008:5008 --name ex8 docker.io/mazurkatarzyna/gpg-ex8:latest
podman run -p 5008:5008 --name ex8 docker.io/mazurkatarzyna/gpg-ex8:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 5008:5008 --name ex8 ghcr.io/mazurkatarzynaumcs/gpg-ex8:latest
podman run -p 5008:5008 --name ex8 ghcr.io/mazurkatarzynaumcs/gpg-ex8:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:5008/sign> używając metody HTTP GET i zapisz odpowiedź do pliku z rozszerzeniem *.zip.
- (c) Rozpakuj pobrane archiwum. W odpowiedzi od serwera otrzymałeś id sesji (jako `session_id`), klucz publiczny GPG (`public_key.asc`), klucz prywatny GPG (`private_key.asc`), oraz słowo do podpisania (w pliku `word.txt`).
- (d) Podpisz plik (`word.txt`) za pomocą pobranego klucza publicznego (`public_key.asc`), tworząc plik `word.txt.gpg`.

Bezpieczeństwo Systemów Komputerowych - GNU Privacy Guard (GPG) | Katarzyna Mazur

- (e) Wyślij request do endpointa <http://127.0.0.1:5008/submit> używając metody HTTP POST i prześlij do serwera id sesji (jako `session_id`) oraz podpisane słowo (jako plik, `signed_file`).

UWAGI:

- Aby zapisać odpowiedź do pliku z rozszerzeniem *.zip, użyj opcji `-o`,
- Aby utworzyć podpis zintegrowany, użyj opcji `--sign`,
- Aby wskazać klucz, którego będziemy używać do podpisywania, sprawdź opcję `-u`,
- Aby za pomocą narzędzia `cURL` wysłać plik do serwera, użyj składni: `-F "plik=@hello.txt"`

```
(.python_env) └─[admin@parrot]-[~]
└─ $curl -s -o pliki.zip -X GET http://localhost:5008/sign
(.python_env) └─[admin@parrot]-[~]
└─ $unzip pliki.zip
Archive: pliki.zip
replace word.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: A
  inflating: word.txt
  inflating: public_key.asc
  inflating: private_key.asc
  inflating: session_id.txt
(.python_env) └─[admin@parrot]-[~]
└─ $cat session_id.txt
db930d716f6f20b1fdda79f98747b62c(.python_env) └─[admin@parrot]-[~]
└─ $gpg --import private_key.asc
gpg: key C833026B005C503C: public key "Sign Challenge <sign@example.com>" imported
gpg: key C833026B005C503C: secret key imported
gpg: Total number processed: 1
gpg:          imported: 1
gpg:    secret keys read: 1
gpg:    secret keys imported: 1
(.python_env) └─[admin@parrot]-[~]
└─ $gpg --sign word.txt -u C833026B005C503C
usage: gpg [options] --sign [filename]
(.python_env) └─[*]-[admin@parrot]-[~]
└─ $gpg --list-keys
/home/admin/.gnupg/pubring.kbx
-----
pub   rsa2048 2025-11-12 [SC] [expires: 2026-11-12]
      82E7B3176D404400AA23BFCC6C4D02D44C4E40A6
uid           [ultimate] Jan Paweł 2 (test) <jp2@watican.com>
sub   rsa2048 2025-11-12 [E] [expires: 2026-11-12]

pub   rsa4096 2017-11-30 [SC] [expired: 2021-11-30]
      7FCA9277C31BB06593495B5D228C64831032DFF0
uid           [ expired] Marek Miśkiewicz (Gmail general authorisation.) <marek.miskiewicz@gmail.com>
uid           [ expired] Marek Miśkiewicz <mmiskiewicz@pjawst.edu.pl>
uid           [ expired] Marek Miśkiewicz <marek.miskiewicz@icloud.com>
```

```
uid          [ unknown] Decrypt Challenge <decrypt@example.com>

pub  rsa2048 2025-11-17 [SCEA]
    BC2C7F144489A0F72566121FC833026B005C503C
uid          [ unknown] Sign Challenge <sign@example.com>

(.python_env) └─[admin@parrot]─[~]
└─ $gpg -u BC2C7F144489A0F72566121FC833026B005C503C --sign word.txt
(.python_env) └─[admin@parrot]─[~]
└─ $curl -X POST http://localhost:5008/submit -F "session_id=$(cat session_id.txt)" -F "signed_file=@word.txt.gpg"
{
  "error": "Missing signed_file",
  "hint": "Wymagany plik signed_file (podpisany plik .gpg)"
}(.python_env) └─[admin@parrot]─[~]
└─ $curl -X POST http://localhost:5008/submit -F "session_id=$(cat session_id.txt)" -F "signed_file=@word.txt.gpg"
{
  "status": "success",
  "message": "🎉 Gratulacje! Poprawnie podpisałeś plik!",
  "word": "lorin",
  "signer": "Sign Challenge <sign@example.com>",
  "signature_valid": true
}(.python_env) └─[admin@parrot]─[~]
└─ $
```